

# MANUAL TÉCNICO

## Centro de Rescate Animal: Gestión de Refugio y Adopciones

| Dato          | Información                                 |
|---------------|---------------------------------------------|
| Nombre        | Wilson Adalberto Aroche Jimenez             |
| Carné         | 202400680                                   |
| Sección       | F                                           |
| Lenguaje      | Java 21 (compatible con Java 17 o superior) |
| IDE utilizado | Visual Studio Code                          |
| Semestre      | Segundo semestre 2026                       |

## Objetivo del manual

Este manual describe la estructura interna, arquitectura, clases, métodos principales, arreglos estáticos, matriz de ubicaciones, persistencia, validaciones y generación de reportes del sistema Centro de Rescate Animal. Su propósito es servir como referencia para comprender, mantener, compilar y explicar técnicamente el proyecto durante su revisión.

## Tecnologías y restricciones

| Elemento                  | Implementación                                                                                       |
|---------------------------|------------------------------------------------------------------------------------------------------|
| Lenguaje                  | Java. El proyecto fue probado con OpenJDK 21.                                                        |
| Interfaz                  | Java Swing construida manualmente mediante código.                                                   |
| Almacenamiento principal  | Arreglos estáticos de objetos y una matriz bidimensional.                                            |
| Persistencia              | Archivos CSV separados por punto y coma (;).                                                         |
| Reportes                  | Archivos HTML con CSS básico generados desde Java.                                                   |
| IDE                       | Visual Studio Code con soporte para Java.                                                            |
| Estructuras no utilizadas | No se utiliza ArrayList, LinkedList, HashMap, List, Queue, Stack ni Vector para la lógica principal. |

## Estructura general del proyecto

```
IPC1_Proyecto1_CentroRescate/  
├── src/  
│   ├── Main.java  
│   ├── interfaz/  
│   │   ├── VentanaLogin.java  
│   │   └── VentanaPrincipal.java  
│   ├── modelo/  
│   │   ├── Animal.java  
│   │   ├── Adoptante.java  
│   │   ├── Solicitud.java  
│   │   ├── Rescate.java  
│   │   ├── Usuario.java  
│   │   └── Bitacora.java  
│   ├── servicio/  
│   │   └── RefugioService.java  
│   ├── persistencia/  
│   │   └── Persistencia.java  
│   ├── reportes/  
│   │   └── GeneradorReportes.java  
│   └── util/  
│       └── FechaUtil.java  
├── datos/  
├── reportes/  
└── README.md
```

La organización separa el modelo de datos, la lógica del sistema, la interfaz gráfica, la persistencia, los reportes y las utilidades. Esto evita concentrar toda la funcionalidad en una sola clase.

## Arquitectura del sistema

El sistema utiliza una organización por capas sencilla. La interfaz recibe las acciones del usuario, RefugioService aplica las reglas y validaciones, Persistencia guarda o recupera la información y GeneradorReportes construye los archivos HTML.

| Capa / paquete | Responsabilidad principal                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| Main           | Crea los componentes principales, carga los datos y abre el inicio de sesión.                                     |
| interfaz       | Construye las ventanas Swing y captura la interacción del usuario.                                                |
| modelo         | Define los objetos que representan animales, adoptantes, solicitudes, rescates, usuarios y registros de bitácora. |
| servicio       | Contiene arreglos, matriz, búsquedas, validaciones y reglas de negocio.                                           |
| persistencia   | Lee y escribe archivos CSV dentro de la carpeta datos/.                                                           |
| reportes       | Genera reportes HTML dentro de la carpeta reportes/.                                                              |
| util           | Genera fechas, fecha/hora y sellos para nombres de archivos.                                                      |
|                |                                                                                                                   |

## Flujo de inicio de la aplicacion.

1. Main ejecuta la interfaz con `SwingUtilities.invokeLater`.
2. Se crea una instancia de `RefugioService`.
3. Se crea `Persistencia` utilizando la carpeta `datos`.
4. `prepararUsuarios()` crea `usuarios.csv` si aún no existe.
5. `cargarTodo()` carga animales, adoptantes, solicitudes, rescates y bitácora.
6. `reconstruirMatrizDesdeAnimales()` reconstruye la ocupación de jaulas a partir de los animales cargados.
7. Se muestra `VentanaLogin`.
8. Después de autenticar, se abre `VentanaPrincipal`.

```
RefugioService servicio = new RefugioService();
Persistencia persistencia = new Persistencia("datos");
persistencia.prepararUsuarios();
persistencia.cargarTodo(servicio);
new VentanaLogin(persistencia, servicio).setVisible(true);
```

## Autenticacion.

La clase `VentanaLogin` solicita usuario y contraseña. `Persistencia.autenticar()` recorre `usuarios.csv` y devuelve un objeto `Usuario` cuando las credenciales coinciden.

Si el archivo no existe, `prepararUsuarios()` crea dos usuarios iniciales: `admin` con rol `ADMINISTRADOR` y `auxiliar` con rol `AUXILIAR`. El rol se muestra en la interfaz y se registra en la bitácora. La versión actual utiliza la misma ventana principal para ambos roles.

## Clases del modelo

| Clase     | Atributos principales                                                                   | Uso                                                               |
|-----------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Animal    | codigo, nombre, especie, edadEstimada, estadoClinico, estadoAdopcion, ubicacion, activo | Representa un animal rescatado y su situación dentro del refugio. |
| Adoptante | dpi, nombre, telefono, correo, direccion, activo                                        | Representa una persona interesada en adoptar.                     |
| Solicitud | codigo, dpiAdoptante, codigoAnimal, fecha, estado                                       | Relaciona un adoptante con un animal.                             |
| Rescate   | codigo, descripcion, ubicacionReporte, prioridad, estado, fecha                         | Representa un reporte de rescate urgente.                         |
| Usuario   | usuario, clave, rol                                                                     | Contiene las credenciales y rol de acceso.                        |
| Bitacora  | fechaHora, usuario, accion                                                              | Registra acciones realizadas en el sistema.                       |

Las clases de modelo incluyen constructores, métodos get, algunos métodos set y, excepto Usuario, métodos toCSV() cuando el objeto debe persistirse en archivos.

## Arreglos estáticos y matriz

La clase RefugioService mantiene toda la información principal en arreglos de tamaño fijo. Cada arreglo posee un contador que indica cuántas posiciones han sido utilizadas.

| Estructura              | Capacidad  | Contenido                    |
|-------------------------|------------|------------------------------|
| Animal[] animales       | 200        | Animales registrados.        |
| Adoptante[] adoptantes  | 150        | Adoptantes registrados.      |
| Solicitud[] solicitudes | 300        | Solicitudes de adopción.     |
| Rescate[] rescates      | 200        | Casos de rescate.            |
| Bitacora[] bitacora     | 1000       | Acciones registradas.        |
| String[][] ubicaciones  | 4 x 8 = 32 | Ocupación de áreas y jaulas. |

```
private final Animal[] animales = new Animal[MAX_ANIMALES];
private final Adoptante[] adoptantes = new Adoptante[MAX_ADOPTANTES];
private final Solicitud[] solicitudes = new Solicitud[MAX_SOLICITUDES];
private final Rescate[] rescates = new Rescate[MAX_RESCATES];
private final Bitacora[] bitacora = new Bitacora[MAX_BITACORA];
private final String[][] ubicaciones =
    new String[FILAS_UBICACION][COLUMNAS_UBICACION];
```

Los contadores totalAnimales, totalAdoptantes, totalSolicitudes, totalRescates y totalBitacora permiten recorrer únicamente la porción utilizada de cada arreglo.

## Lógica principal: RefugioService

RefugioService centraliza la lógica de negocio. La interfaz no modifica directamente los arreglos: llama a métodos del servicio y recibe 'OK' o un mensaje de error.

## Gestión de animals

| Método                     | Función                                                                                       |
|----------------------------|-----------------------------------------------------------------------------------------------|
| registrarAnimal(...)       | Valida datos, especie, edad, duplicados y capacidad; crea un animal DISPONIBLE y SIN_ASIGNAR. |
| buscarAnimalPorCodigo(...) | Recorre el arreglo de animales hasta encontrar el código solicitado.                          |
| buscarAnimales(...)        | Busca por código, nombre, especie o estado y devuelve un arreglo de resultados.               |
| editarAnimal(...)          | Actualiza los datos de un animal activo.                                                      |
| eliminarAnimalLogico(...)  | Libera su ubicación, cambia activo a false y el estado de adopción a ELIMINADO.               |

## Gestión de adoptantes

| Método                  | Función                                                           |
|-------------------------|-------------------------------------------------------------------|
| registrarAdoptante(...) | Valida campos, evita DPI duplicado y agrega el objeto al arreglo. |
| buscarAdoptante(...)    | Busca un adoptante mediante DPI.                                  |
| editarAdoptante(...)    | Actualiza nombre, teléfono, correo y dirección.                   |

## Solicitudes de adopcion

Una solicitud nueva se crea con estado PENDIENTE. Para registrarla deben existir tanto el adoptante como el animal, y el animal debe estar DISPONIBLE.

| Método                        | Función                                                                                |
|-------------------------------|----------------------------------------------------------------------------------------|
| registrarSolicitud(...)       | Valida código, adoptante, animal y disponibilidad.                                     |
| buscarSolicitud(...)          | Busca una solicitud por su código.                                                     |
| cambiarEstadoSolicitud(...)   | Permite pasar una solicitud pendiente a APROBADA o RECHAZADA.                          |
| rechazarOtrasSolicitudes(...) | Cuando una adopción se aprueba, rechaza otras solicitudes pendientes del mismo animal. |

Cuando una solicitud se aprueba, el estado del animal cambia a ADOPTADO y cualquier ubicación física que tuviera asignada se libera. También se impide procesar nuevamente una solicitud que ya no esté PENDIENTE.

## Rescates urgentes

| Método                | Función                                                  |
|-----------------------|----------------------------------------------------------|
| registrarRescate(...) | Crea un rescate con estado ACTIVO y valida la prioridad. |
| buscarRescate(...)    | Busca por código.                                        |
| atenderRescate(...)   | Cambia un rescate ACTIVO a ATENDIDO.                     |

Las prioridades válidas implementadas son BAJA, MEDIA, ALTA y CRITICA.

## Matriz de ubicaciones

El refugio utiliza una matriz String de 4 filas por 8 columnas. Las filas representan áreas y las columnas jaulas. Una posición libre contiene la cadena LIBRE; una ocupada contiene el código del animal.

```
FILAS_UBICACION = 4
COLUMNAS_UBICACION = 8

A1-J1  A1-J2  ...  A1-J8
A2-J1  A2-J2  ...  A2-J8
A3-J1  A3-J2  ...  A3-J8
A4-J1  A4-J2  ...  A4-J8
```

| Método                           | Descripción                                                                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| asignarUbicacion(...)            | Comprueba que el animal exista, que fila/columna sean válidas, que la celda esté LIBRE y que el animal no tenga otra ubicación. |
| liberarUbicacion(...)            | Devuelve la celda a LIBRE y cambia la ubicación del animal a SIN_ASIGNAR.                                                       |
| liberarUbicacionDeAnimal(...)    | Recorre toda la matriz para liberar cualquier celda que contenga el código indicado.                                            |
| contarEspaciosOcupados()         | Cuenta las celdas diferentes de LIBRE.                                                                                          |
| capacidadTotal()                 | Devuelve $4 \times 8 = 32$ .                                                                                                    |
| reconstruirMatrizDesdeAnimales() | Al cargar el programa, reconstruye la matriz usando ubicaciones con formato A1-J1 a A4-J8.                                      |

La interfaz solicita área 1-4 y jaula 1-8. Antes de llamar al servicio resta 1 a ambos valores, porque los índices internos de la matriz comienzan en cero.

## Persistencia de datos

Persistencia administra la carpeta datos/. El programa utiliza File, FileReader, FileWriter, BufferedReader y PrintWriter para leer y escribir información.

Los archivos usan punto y coma (;) como separador. Al guardar, cada clase convierte sus atributos a una línea mediante toCSV().

| Archivo         | Formato de cada línea                                                    |
|-----------------|--------------------------------------------------------------------------|
| usuarios.csv    | usuario;clave;rol                                                        |
| animales.csv    | codigo;nombre;especie;edad;estadoClinico;estadoAdopcion;ubicacion;activo |
| adoptantes.csv  | dpi;nombre;telefono;correo;direccion;activo                              |
| solicitudes.csv | codigo;dpiAdoptante;codigoAnimal;fecha;estado                            |
| rescates.csv    | codigo;descripcion;ubicacionReporte;prioridad;estado;fecha               |
| bitacora.csv    | fechaHora;usuario;accion                                                 |

## Carga de datos

cargarTodo() llama de forma secuencial a cargarAnimales, cargarAdoptantes, cargarSolicitudes, cargarRescates y cargarBitacora. Después reconstruye la matriz de ubicaciones.

Cada método recorre el archivo correspondiente línea por línea, divide el contenido con split(';') y crea el objeto de modelo necesario.

## Guardado de datos

guardarTodo() sobrescribe los archivos CSV con el contenido actual de los arreglos. VentanaPrincipal llama este método al presionar Guardar datos, al generar reportes y al cerrar la aplicación.

```
public void guardarTodo(RefugioService s) throws IOException {  
    guardarAnimales(s);  
    guardarAdoptantes(s);  
    guardarSolicitudes(s);  
    guardarRescates(s);  
    guardarBitacora(s);  
}
```

## Concideracion de formato.

Debido a que el separador utilizado es el punto y coma, se recomienda no introducir este carácter dentro de los campos almacenados. La acción de bitácora sí reemplaza punto y coma por coma antes de guardarse.

# Generación de reportes HTML

GeneradorReportes crea automáticamente la carpeta reportes/ cuando no existe. Cada reporte utiliza un nombre con fecha y hora para evitar sobrescribir archivos anteriores.

```
nombre + "_" + FechaUtil.selloArchivo() + ".html"
```

Ejemplo:

```
animales_20260905_091918.html
```

| Método                 | Contenido                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------|
| reporteAnimales(...)   | Tabla con código, nombre, especie, edad, estado clínico, adopción, ubicación y estado activo. |
| reporteAdopciones(...) | Tabla con código de solicitud, DPI, animal, fecha y estado.                                   |
| reporteOcupacion(...)  | Cantidad de espacios ocupados, capacidad total y matriz de 4 x 8.                             |
| reporteBitacora(...)   | Fecha/hora, usuario y acción registrada.                                                      |

El método inicio() escribe la estructura HTML, codificación UTF-8 y estilos CSS básicos. El método e() escapa caracteres especiales como &, <, > y comillas antes de incorporarlos al HTML.

## Interfaz gráfica

La interfaz está programada manualmente con Swing. No se utilizan diseñadores visuales.

| Clase            | Componentes y responsabilidad                                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| VentanaLogin     | JFrame, JPanel, JTextField, JPasswordField, JButton y JOptionPane. Realiza el acceso al sistema.                                       |
| VentanaPrincipal | JFrame con JTabbedPane. Contiene módulos de animales, adoptantes, solicitudes, rescates, ubicaciones, reportes, bitácora y estudiante. |
| Tablas           | DefaultTableModel no editable + JTable para mostrar los datos de cada módulo.                                                          |
| Formularios      | JOptionPane con JTextField y JComboBox para registrar o modificar información.                                                         |

El método refrescarTodo() actualiza las tablas de animales, adoptantes, solicitudes, rescates, ubicaciones y bitácora después de las operaciones que modifican información.

## Validaciones y manejo de errores

| Validación                | Comportamiento implementado                                                                               |
|---------------------------|-----------------------------------------------------------------------------------------------------------|
| Campos vacíos             | Devuelve 'Hay campos vacíos.' o solicita completar usuario/contraseña.                                    |
| Código de animal repetido | Impide registrar otro animal con el mismo código.                                                         |
| Edad                      | Solo se acepta un valor entre 0 y 40.                                                                     |
| Especie                   | Solo Perro o Gato.                                                                                        |
| DPI repetido              | Impide registrar un adoptante con DPI existente.                                                          |
| Solicitud repetida        | Impide códigos de solicitud duplicados.                                                                   |
| Disponibilidad            | Solo permite solicitar adopción de animales DISPONIBLES.                                                  |
| Solicitud ya procesada    | Impide aprobar o rechazar nuevamente una solicitud no PENDIENTE.                                          |
| Prioridad de rescate      | Solo BAJA, MEDIA, ALTA o CRITICA.                                                                         |
| Ubicación                 | Valida límites de matriz, celda ocupada y animal ya ubicado.                                              |
| Entrada numérica          | La interfaz captura NumberFormatException al ingresar área o jaula no numérica.                           |
| Búsqueda sin resultado    | Muestra un aviso si el arreglo de resultados queda vacío.                                                 |
| Errores de archivo        | Las ventanas muestran mensajes cuando ocurre una excepción de lectura, guardado o generación de reportes. |

## Bitacora tecnica .

RefugioService.registrarBitacora() crea objetos Bitacora con fecha/hora, usuario actual y acción. Se registran eventos como inicio de sesión, registro o edición de animales, adopciones, rescates, asignación y liberación de espacios y generación de reportes.

La capacidad máxima de la bitácora es de 1000 registros. Al llegar a este límite, el método deja de agregar nuevas entradas.

## Manejo de fechas

FechaUtil utiliza LocalDateTime y DateTimeFormatter. Se manejan tres formatos:

| Método            | Formato             | Uso                                |
|-------------------|---------------------|------------------------------------|
| fechaActual()     | dd/MM/yyyy          | Solicitudes y rescates.            |
| fechaHoraActual() | dd/MM/yyyy HH:mm:ss | Bitácora y encabezado de reportes. |
| selloArchivo()    | yyyyMMdd_HHmmss     | Nombre único de reportes HTML.     |

# Compilación y ejecución en Visual Studio Code

El proyecto fue probado en Windows con OpenJDK 21. Desde PowerShell, ubicado en la carpeta raíz del proyecto, puede compilarse de la siguiente forma:

```
New-Item -ItemType Directory -Force out
javac -d out (Get-ChildItem -Recurse src -Filter *.java).FullName
java -cp out Main
```

Cuando se modifica código fuente, se recomienda volver a compilar. Para eliminar una compilación anterior y evitar ejecutar archivos .class desactualizados:

```
Remove-Item -Recurse -Force out -ErrorAction SilentlyContinue
New-Item -ItemType Directory -Force out
javac -d out (Get-ChildItem -Recurse src -Filter *.java).FullName
java -cp out Main
```

## Pruebas funcionales realizadas

| No. | Prueba                              | Resultado                               |
|-----|-------------------------------------|-----------------------------------------|
| 1   | Código de animal duplicado          | Aprobada                                |
| 2   | Campos vacíos                       | Aprobada                                |
| 3   | Búsqueda de animal inexistente      | Aprobada después de agregar aviso       |
| 4   | Edición de animal                   | Aprobada                                |
| 5   | Eliminación lógica                  | Aprobada                                |
| 6   | DPI de adoptante duplicado          | Aprobada                                |
| 7   | Rechazo de solicitud                | Aprobada                                |
| 8   | Solicitud ya procesada              | Aprobada después de corregir validación |
| 9   | Intento de ocupar una jaula ocupada | Aprobada                                |
| 10  | Área o jaula fuera de rango         | Aprobada                                |
| 11  | Persistencia después de reiniciar   | Aprobada                                |

Durante las pruebas se detectaron dos mejoras: mostrar un aviso al no encontrar animales y evitar que una solicitud ya procesada apareciera como una operación exitosa. Ambas validaciones fueron corregidas y se recompiló el proyecto.

## Decisiones de diseño

- Se utilizaron arreglos estáticos y matrices debido a la restricción académica del proyecto.
- RefugioService concentra las reglas para evitar duplicar lógica entre las ventanas.
- La interfaz se separó de la persistencia y los reportes para facilitar la lectura y mantenimiento.
- La ubicación se guarda en el Animal con formato A#-J# y se reconstruye en una matriz al iniciar.
- La eliminación de animales es lógica mediante el atributo activo.
- Los reportes utilizan HTML sencillo para poder abrirse en cualquier navegador.

## Consideraciones para mantenimiento

- Si se aumenta la cantidad máxima de registros, modificar las constantes MAX\_\* en RefugioService.
- Si cambia el tamaño físico del refugio, actualizar FILAS\_UBICACION, COLUMNAS\_UBICACION y la validación del formato de ubicación.
- Si se agregan campos a un modelo, actualizar constructor, getters/setters, toCSV(), carga/guardado, tablas Swing y reportes.
- Mantener el mismo orden de campos entre toCSV() y los métodos de carga.
- Después de modificar el código, eliminar o recompilar la carpeta out antes de ejecutar.
- No introducir estructuras del Java Collections Framework en la lógica principal si se mantiene la restricción del proyecto.

## Puntos claves para la defensa

Explicar por qué se usan arreglos estáticos y cómo funcionan los contadores total\*.

- Explicar cómo se recorre la matriz 4 x 8 y cómo se valida una ubicación.
- Mostrar el flujo desde Main hasta VentanaLogin y VentanaPrincipal.
- Explicar cómo se guarda y recupera un Animal mediante toCSV() y Persistencia.
- Explicar la diferencia entre eliminación lógica y eliminación física.
- Explicar qué sucede al aprobar una solicitud y cómo cambia el estado del animal.
- Explicar cómo se genera un reporte HTML y cómo se construye su nombre.
- Poder modificar una validación o un texto de la interfaz y recompilar el proyecto.

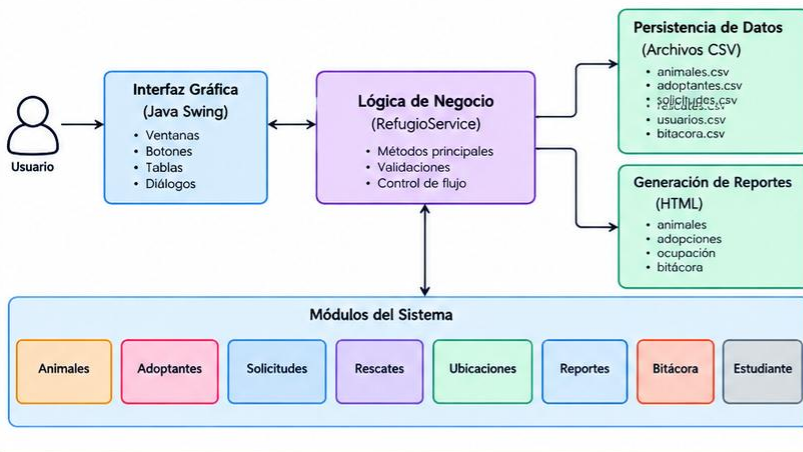
## Conclusión técnica

El proyecto está dividido en componentes con responsabilidades específicas y utiliza Java Swing, arreglos estáticos, matrices, archivos CSV y reportes HTML. La lógica de negocio se concentra en RefugioService, mientras que la interfaz, persistencia y reportes se mantienen separados.

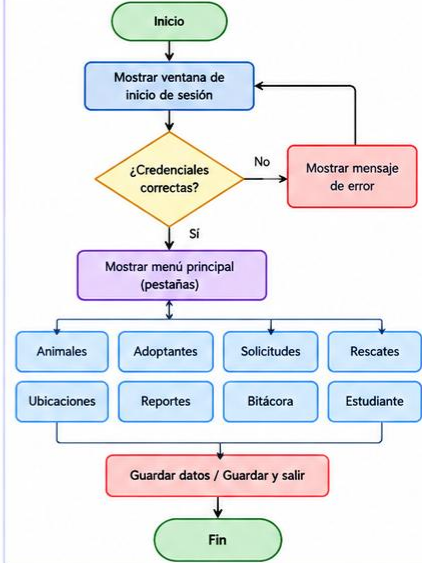
Esta estructura permite comprender el flujo del sistema y realizar cambios sin modificar todos los archivos al mismo tiempo. El manual describe la versión probada durante el desarrollo y las validaciones verificadas en las pruebas funcionales.

# Digrama.

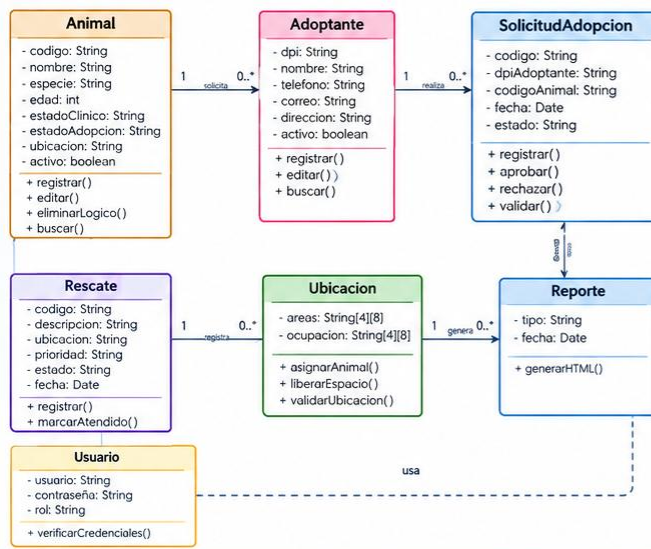
## 1. Diagrama de Arquitectura del Sistema



## 2. Diagrama de Flujo del Menú Principal



## 3. Diagrama de Clases (Modelo)

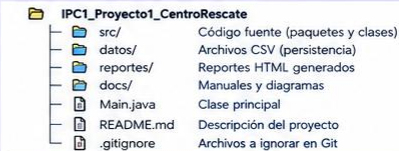


## 4. Matriz de Ubicaciones (4 áreas x 8 jaulas)

|        | Jaula 1 | Jaula 2 | Jaula 3 | Jaula 4 | Jaula 5 | Jaula 6 | Jaula 7 | Jaula 8 |
|--------|---------|---------|---------|---------|---------|---------|---------|---------|
| Área 1 | LIBRE   | LIBRE   | A1-J3   | LIBRE   | LIBRE   | LIBRE   | LIBRE   | LIBRE   |
| Área 2 | LIBRE   | LIBRE   | LIBRE   | B2-J4   | LIBRE   | LIBRE   | LIBRE   | LIBRE   |
| Área 3 | LIBRE   | C3-J2   | LIBRE   | LIBRE   | LIBRE   | LIBRE   | LIBRE   | LIBRE   |
| Área 4 | LIBRE   | LIBRE   | LIBRE   | LIBRE   | D4-J5   | LIBRE   | LIBRE   | LIBRE   |

LIBRE = Jaula disponible      A1-J3 = Animal asignado (ejemplo)

## 5. Estructura de Carpetas del Proyecto



Nota: Los diagramas muestran la estructura general del sistema. La implementación real puede incluir más clases y detalles según el desarrollo del proyecto.